CheckInOut – Hostel Management System

CS 432 – Databases (Course Project – Assignment 2)

Module B: Optimisation Implementation Report

Semester II (2025–2026)

Date: 22 March 2026

Instructor: Dr. Yogesh K. Meena

Team Members & Contributions

Vipul Sunil Patil – Schema design, indexing strategy, benchmarking
Sai – Security implementation, RBAC, report writing
Daksh Jain – API profiling, performance data collection
Daksh Dave – Data seeding, EXPLAIN analysis, documentation
Harshit – Index creation, query optimisation, verification

GitHub Repository:
https://github.com/Harshitiitgn/CS432_Track1_Group10_A2

Analysis Notebook (Plots, Python Code & Extended Benchmarking):
https://colab.research.google.com/drive/1Qwuu3ec83b0pBhV1z-TQZXes422Njoyk?usp=sharing

Video Link:
https://www.youtube.com/watch?v=Xnf_NpJOGJw

© Indian Institute of Technology Gandhinagar

Abstract

This report documents the optimisation implementation for the CheckInOut Hostel Management System (CS 432 – Databases, Module B). It covers four deliverables: (1) **Schema Design** – how core authentication tables are separated from the 12 project-specific hostel tables to maintain 3NF integrity without duplication; (2) **Security** – how HttpOnly cookie-based session validation prevents XSS/data leaks, how RBAC enforces Admin vs. Regular User permissions, and how IDOR vulnerabilities are eliminated; (3) **Indexing Strategy** – which columns were indexed, the B+Tree traversal behaviour observed in `EXPLAIN QUERY PLAN` output, and the elimination of in-memory sort operations; and (4) **Performance Benchmarking** – quantitative API response time measurements before and after indexing with full statistical analysis. Results: 25 of 33 endpoints (76%) improved, maximum mean improvement 47.1%, average improvement 29.4%, rows examined per dashboard query reduced by $> 99\%$.

Keywords

Database Optimisation, SQL Indexing, EXPLAIN Query Plan, RBAC, HttpOnly Cookies, IDOR Prevention, Hostel Management

1 Introduction

The CheckInOut Hostel Management System (Assignment 1) established a 12-table relational schema covering student check-in/check-out, room allocation, furniture inventory, complaint handling, visitor logging, maintenance requests, and QR-based security auditing. Assignment 2 – Module B builds on this schema with three goals:

(1) **Security hardening:** Add secure authentication and authorization by implementing HttpOnly cookie–based session handling and a Role-Based Access Control (RBAC) layer without duplicating hostel domain data in credential tables. Introduce a **core system table, Users**, which stores user credentials including securely hashed (not plain or reversible "encrypted") passwords.
(2) **Performance optimisation:** Apply SQL indexes to the most-queried columns and measure the quantitative impact.
(3) **Quantitative evidence:** Record API response times before and after indexing, analyse them statistically (Mean, Robust Mean, Median, P95), and verify execution plan changes via `EXPLAIN QUERY PLAN`.

The backend uses **Node.js + Express** with **SQLite**. The seed dataset was expanded to **~900+ rows** across all tables (50 Members, 120 Allocations, 100 Complaints, 50 Maintenance, 40 visitors proportional elsewhere) to simulate realistic institutional load.

2 Schema Design

2.1 Design Principle: 3NF and Separation of Concerns

The database follows **Third Normal Form (3NF)** to eliminate data redundancy. It is partitioned into two layers: **Core System Tables** for authentication/access control, and **Project-Specific Tables** for all hostel business domain data. No hostel attribute ever enters an authentication table; no credential ever enters a hostel table.

2.2 Entity Segregation

Core entities (`Member`, `Room`, `Hostel`) are separated from operational bridging tables (`Allocation`, `Complaint`, `Visitor`). This prevents any single relation from carrying both identity data and transactional data, keeping the schema clean and extensible.

2.3 Natural vs. Surrogate Keys

The primary key of the `Member` table was migrated from an abstract `MemberID` integer to the natural `IdentificationNumber`. This eliminates an unnecessary indirection layer across `Allocation`, `Complaint`, `Visitor`, `FeePayment`, and `MaintenanceRequest` – all of which now join directly on `IdentificationNumber` without an intermediate lookup step.

2.4 Core System Tables

Table 1: Core Authentication & Session Tables

Table	Key Columns	Purpose
`Users`	`userId, username, password_hash, role`	Login credentials. role ∈ {Admin, Regular}
`Sessions`	`sessionId, userId, token, expiresAt`	Active HttpOnly cookie session registry
`UserMember Mapping`	`mappingId, userId, memberId`	1:1 bridge linking login account to hostel Member record

2.5 Project-Specific Tables

The 12 hostel-domain tables carry zero authentication data.

Table 2: Hostel Domain Tables

Table	Type	Purpose
`Member`	Core	Resident data
`Hostel`	Core	Hostel blocks
`Room`	Core	Room details
`RoomType`	Lookup	Room categories
`Allocation`	Bridge	Stay records
`FurnitureType`	Lookup	Furniture types
`FurnitureItem`	Core	Furniture tracking
`Complaint`	Core	Complaint records
`ComplaintCategory`	Lookup	Complaint types
`Visitor`	Core	Visitor logs
`QRScanLog`	Log	QR scan logs
`MaintenanceRequest`	Core	Maintenance tasks

,
2026.

Table 3: Project-Specific Tables – References, Cascade Behaviour, and Role

Table	References	Cascade Policy	Domain Role
Hostel	–	Root entity	Building-level information and capacity
Room	Hostel	CASCADE on HostelID	Individual room with occupancy tracking
Member	–	Identity anchor	Student/resident profile (Name, Age, Email, etc.)
Allocation	Member, Room	CASCADE on both	Check-in/check-out bridge entity; preserves history
Staff	Hostel	CASCADE on HostelID	Warden and staff records
FeeCategory	–	Lookup table	Fee type definitions
FeePayment	Member, FeeCategory	CASCADE on Member	Payment history per member
Complaint	Member	CASCADE on Member	User-reported issues with severity tracking
Visitor	Member	CASCADE on Member	Guest entry/exit log
MaintenanceRequest	Member, Room	CASCADE on both	Work-order tracking
FurnitureItem	Member, Room	CASCADE on both	Individual asset tracking with condition status
QRScanLog	Member, Staff	CASCADE on both	Security audit trail for QR-based gate access

2.6 Lookup Tables for Extensibility

Entities like `RoomType`, `ComplaintCategory`, and `FurnitureType` act as strictly-defined lookup tables with controlled columns (`IsAC`, `BaseCapacity`, `TypeName`). This prevents free-text typos and data anomalies in core entity tables, and allows new categories to be added without schema changes.

2.7 Constraint Enforcement

2.8 Integrity Guarantees

- **No credential duplication:** `password_hash` and `role` exist only in `Users`; the `Member` table contains zero authentication fields.
- **Referential integrity:** All FK relationships use `ON DELETE CASCADE` and `ON UPDATE CASCADE` where appropriate, with `RESTRICT` for critical active-dependency boundaries.
- **1:1 account–member link:** `UserMemberMapping` carries a UNIQUE constraint on both `userId` and `memberId`.
- **Automatic cleanup:** Deleting a Member row cascades to all eight child tables in a single transaction – no orphaned rows, no manual cleanup required.
- **Single active allocation:** A UNIQUE constraint on (`MemberID`, `AllocationStatus='Active'`) ensures no member has two simultaneous active room assignments.

3 Security

The backend was significantly overhauled with a three-layer security model: **HttpOnly cookie-based session validation**, **Role-Based Access Control (RBAC)** middleware with IDOR prevention, and **dual-layer audit logging**.

3.1 Session Validation via HttpOnly Cookies

The authentication layer was migrated from **localStorage-based JWT storage** to **HttpOnly, Secure Cookies** to significantly improve client-side security. Tokens stored in `localStorage` are directly accessible to JavaScript, making them highly susceptible to **Cross-Site Scripting (XSS)** attacks. In contrast, HttpOnly cookies are managed entirely by the browser and cannot be accessed, modified, or exfiltrated through any client-side script.

The backend issues the JWT using `res.cookie()` with strict security attributes such as `HttpOnly`, `Secure`, and `SameSite`, ensuring safe transmission and restricted cross-origin usage. This design shifts session handling responsibility to the browser, reducing the attack surface and enforcing a more robust trust boundary between client and server.

- The `authenticateToken` middleware explicitly extracts `req.cookies.token` – not an Authorization header.
- Unauthorized requests are dropped *instantly* before any database query is executed.
- Token expiry is validated on every request; expired sessions receive **401 Unauthorized**.
- On success, `req.user = { userId, identificationNumber, role }` is attached for downstream RBAC checks.

Listing 1: authenticateToken Middleware – HttpOnly Cookie Extraction

```
// authenticateToken middleware (simplified)

function authenticateToken(req, res, next) {
    // Read JWT from HttpOnly Cookie (set at login)
         instead of Authorization header
    const token = req.cookies?.token;

    if (!token) return res.sendStatus(401);

    jwt.verify(token, JWT_SECRET, (err, user) => {
        if (err) return res.sendStatus(403);
        // req.user = user;
        req.user = {
            userId: decoded.userId,
            identificationNumber: decoded.
                 identificationNumber,
            role: decoded.role // 'Admin' | 'Regular'
        }
        next();
    });
}
```

Table 4: Constraint Specifications Across All Tables

Constraint Type	Examples	Purpose
CHECK	`CheckOutDate >= CheckInDate`, `Age > 0`, `CurrentOccupancy <= MaxCapacity`	Validate data ranges and logical relationships
UNIQUE	`QRCode`, `Email`, `HostelID + RoomNumber`, `IdentificationNumber`	Prevent duplicate combinations
NOT NULL	`IdentificationNumber`, `RoomNumber`, `CheckInDate`, `HostelID`	Mandatory columns (3–8 per table)
ON DELETE RESTRICT	Member with active Allocation	Prevent deletion of records with active dependencies
ON DELETE CASCADE	FeePayment → Member; Complaint → Member	Automatic child-record cleanup on parent deletion
ENUM / Controlled	`RoomStatus`, `AllocationStatus`, `ComplaintStatus`, `Gender`	Prevent invalid free-text entries

3.2 RBAC: Admin vs. Regular User

Roles are enforced by two distinct middleware functions applied at the route level:

Listing 2: requireOwnershipOrAdmin – IDOR Prevention Middleware

```
// IDOR Prevention middleware ( simplified )

function requireOwnershipOrAdmin(req, res, next) {
    if (!req.user) return res.sendStatus(401);

    // Admin passes through immediately
    if (req.user.role === 'Admin') return next();

    // Block horizontal privilege escalation
    if (req.params.id && req.user.identificationNumber
          === req.params.id) {
        return next();
    }

    return res.status(403).json({
        error: 'Access denied: You can only view your
             own data.'
    });
}
```

IDOR Protection Example

Authorized Access: `GET /api/complaints/member/2023001`
Blocked Access: `GET /api/complaints/member/2023002`

Result: 403 Forbidden returned for unauthorized requests, preventing horizontal privilege escalation.

See Table 5 for RBAC enforcement details.

3.3 Audit Logging – Two-Layer Detection

Refer to Table 6 for detailed audit log structure and event classification.

4 Indexing Strategy

4.1 Pre-existing Indexes and Baseline State

Before any manual indexes were created, we first audited the database using `sqlite3 .indices` to understand what auto-indexes SQLite had already generated from schema constraints. We also confirmed the row counts that define the scan cost:

Listing 3: Terminal: Row Count Verification and Pre-existing Auto-Index Audit

```
$ sqlite3 hostel.db '.indices'
sqlite_autoindex_ComplaintCategory_1
     sqlite_autoindex_Member_3
sqlite_autoindex_FeeCategory_1
     sqlite_autoindex_Member_4
sqlite_autoindex_FurnitureType_1
     sqlite_autoindex_RoomType_1
sqlite_autoindex_Hostel_1
     sqlite_autoindex_Room_1
sqlite_autoindex_Member_1
     sqlite_autoindex_Room_2
sqlite_autoindex_Member_2
     sqlite_autoindex_Users_1
```

All pre-existing indexes are **auto-generated** by SQLite to enforce `UNIQUE` constraints on primary keys and unique columns. Critically, **none of them target the heavy query patterns** used by the API – no index existed on
`Allocation.CheckInDate`,
`Complaint.IdentificationNumber`,
`Visitor.IdentificationNumber`,
`FeePayment.MemberID`, or
`MaintenanceRequest.RequestedBy`.
This confirmed that all high-traffic JOIN and WHERE operations were performing full table scans.

4.2 Why and Where We Indexed

Targeted composite and single-column indexes were applied based on the API's SQL query signatures. Each index was selected by inspecting columns used in `WHERE` clauses (filtering), `JOIN` conditions, and `ORDER BY` clauses (sorting).

To identify the exact columns requiring indexing, we performed a thorough analysis of real application logs and benchmark results. We examined the `access.log` (Morgan logs) and detailed performance benchmark output (**`Before.log` and `After.log`**) generated from our testing scripts. This analysis revealed the **most frequently used** and **most time-consuming** endpoints in the CheckInOut Hostel Management System.

The top time-taking and most-used endpoints (sorted by RobustMean) were:

- `POST /api/auth/login` (RobustMean: 88.45 ms) – critical for every user session.

Table 5: RBAC Middleware – Role Comparison and Applied Endpoints

Middleware	Applied To	Behaviour	Example Endpoints

`requireAdmin`	Admin-only global endpoints	Completely blocks non-Admin execution. Returns **403 Forbidden** for Regular Users regardless of resource.	`GET /api/stats`
			`POST /api/members`
			`POST /api/rooms`
			`DELETE /api/members/:id`

`requireOwnership OrAdmin`	Member-scoped endpoints	Inspects `req.user.identificationNumber` against `req.params.id`. Blocks User A from accessing User B's data. Prevents IDOR vulnerabilities.	`GET /api/complaints/member/:id`
			`GET /api/visitors/member/:id`
			`GET /api/allocations/member/:id`
			`GET /api/fees/member/:id`

Table 6: Dual-Layer Audit Logging Strategy

Layer	Mechanism	What Is Logged	Tamper Detection
Application	Morgan → `audit.log`	Timestamp, userId, HTTP method, endpoint, IP address for every POST/PUT/PATCH/DELETE request.	Provides API-level trail. Direct DB writes do **not** appear here.
Database	SQLite AFTER triggers on all 12 tables → `AuditLog` table	Timestamp, userId, action (INSERT/UPDATE/DELETE), table_name for every SQL data-change statement.	Direct DB writes bypassing the API **do** appear here but not in `audit.log` – the discrepancy flags unauthorized access.

- `GET /api/members` and `GET /api/members/:id` (RobustMean: 13.87–20.06 ms) – heavily used in Member Portfolio and Admin dashboards.
- `GET /api/allocations` and `GET /api/allocations/member/:id` (RobustMean: 13.52–17.47 ms) – core check-in/out history queries.
- `GET /api/rooms` (RobustMean: 13.39 ms) – room status and availability checks.
- `GET /api/complaints` and `GET /api/complaints/member/:id` (RobustMean: 12.71 ms) – complaint tracking.
- `GET /api/fees` and `GET /api/fees/member/:id` (RobustMean: 10.58 ms) – fee dues and payment history.
- `GET /api/visitors` and `GET /api/maintenance` (RobustMean: 9.77–10.03 ms) – visitor logs and maintenance requests.

(A more detailed version of this can be found in Section 5 (Performance Benchmarking).

These endpoints dominate both regular user (Member Portfolio) and admin workflows. Full table scans on large tables (150+ Members, 120+ Allocations, 100+ Complaints) were the main bottleneck before indexing.

The indexes were chosen specifically to target columns appearing in the slowest and most-used queries identified from our logs and benchmark data. The result is a dramatic reduction in full table scans and a clear performance boost across the entire system.

For complete raw logs, statistical code, and interactive graphs, refer to the attached Colab notebook: **benchmark_analysis.ipynb**. The notebook contains detailed analysis of the endpoints discussed above and all relevant data can also be seen in **Section 5 (Performance Benchmarking)**.

- Complete before/after comparison tables
- Line graphs and box plots
- 95th percentile and outlier detection
- IQR-based Robust Mean calculation
- EXPLAIN QUERY PLAN outputs

4.3 SQL CREATE INDEX Statements

Targeted indexes were created on key columns to optimize frequently executed queries. For instance,

`idx_allocations_full` enables `/api/allocations` to fetch pre-sorted records efficiently, while

`idx_complaints_full` allows direct retrieval of a member's complaints via an index seek.

Other indexes like `idx_members_list`, `idx_rooms_status`,

`idx_visitors_full`, and `idx_maintenance_full` target columns used in WHERE filters or FK joins, improving query performance.

```sql
CREATE INDEX IF NOT EXISTS idx_members_list
    ON Member(IdentificationNumber, Email);

CREATE INDEX IF NOT EXISTS idx_allocations_full
    ON Allocation(CheckInDate DESC,
    IdentificationNumber, RoomID);

CREATE INDEX IF NOT EXISTS idx_rooms_status
    ON Room(HostelID);

CREATE INDEX IF NOT EXISTS idx_complaints_full
    ON Complaint(IdentificationNumber);

CREATE INDEX IF NOT EXISTS idx_visitors_full
    ON Visitor(IdentificationNumber);

CREATE INDEX IF NOT EXISTS idx_maintenance_full
    ON MaintenanceRequest(RequestedBy);
```

Listing 4: requireOwnershipOrAdmin – IDOR Prevention Middleware

Table 7: Index Definitions – Columns, Tables, and Justification

Index Name	Table	Columns Indexed	Query Pattern & Justification
`idx_members_list`	Member	`IdentificationNumber, Email`	Appears in nearly every WHERE clause. Composite covers both login-mapping and direct member-search queries.
`idx_allocations_full`	Allocation	`CheckInDate DESC, IdentificationNumber, RoomID`	GET /api/allocations orders by `CheckInDate DESC` while joining two FK columns. Composite descending index eliminates `USE TEMP B+TREE FOR ORDER BY` entirely.
`idx_rooms_status`	Room	`HostelID`	Room dashboard and types endpoints always filter by `HostelID`. Converts full room-table scan to a direct seek.
`idx_complaints_full`	Complaint	`IdentificationNumber`	Student dashboard runs `WHERE IdentificationNumber = ?` over 100+ complaints. Verified via terminal EXPLAIN (see below).
`idx_visitors_full`	Visitor	`IdentificationNumber`	Visitor list per member; same lookup pattern as complaints.
`idx_maintenance_full`	MaintenanceRequest	`RequestedBy`	Maintenance requests filter on `RequestedBy` for both member and admin dashboards.
`idx_fees_full`	FeePayment	`MemberID, PaymentDate DESC`	Queries filter by MemberID AND sort by date. Composite covering index satisfies both predicates in one B+Tree pass.
`idx_furniture_member`	FurnitureItem	`MemberID`	Furniture endpoints filter by MemberID for member-scoped views.

4.4 EXPLAIN QUERY PLAN Analysis

This section demonstrates how indexing optimizations affect SQLite query execution plans. We examine two critical endpoints: `/api/allocations` and `/api/complaints/member/:id`.

4.4.1 Allocations Endpoint The query joins `Allocation` with `Member`, `Room`, and `Hostel`, ordering results by `CheckInDate DESC`.

SQL Statement:

```sql
SELECT a.*, m.Name AS MemberName, m.ContactNumber,
       r.RoomNumber, h.Name AS HostelName
FROM   Allocation a
JOIN   Member m ON a.IdentificationNumber = m.
     IdentificationNumber
JOIN   Room   r ON a.RoomID = r.RoomID
JOIN   Hostel h ON r.HostelID = h.HostelID
ORDER  BY a.CheckInDate DESC;
```

Listing 5: Query for /api/allocations EXPLAIN

EXPLAIN QUERY PLAN Before Indexing:

```
% sqlite3 hostel.db 'EXPLAIN QUERY PLAN SELECT a.*, m.
    Name as MemberName, m.ContactNumber, r.RoomNumber,
    h.Name as HostelName FROM Allocation a JOIN Member
    m ON a.IdentificationNumber = m.
    IdentificationNumber JOIN Room r ON a.RoomID = r.
    RoomID JOIN Hostel h ON r.HostelID = h.HostelID
    ORDER BY a.CheckInDate DESC;'

QUERY PLAN
|--SCAN a
|--SEARCH m USING INDEX sqlite_autoindex_Member_3 (
    IdentificationNumber=?)
|--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
`--USE TEMP B+TREE FOR ORDER BY
```

EXPLAIN QUERY PLAN After Adding Composite Index `idx_allocations_full`:

```
% sqlite3 hostel.db 'EXPLAIN QUERY PLAN SELECT a.*, m.
    Name as MemberName, m.ContactNumber, r.RoomNumber,
    h.Name as HostelName FROM Allocation a JOIN Member
    m ON a.IdentificationNumber = m.
    IdentificationNumber JOIN Room r ON a.RoomID = r.
    RoomID JOIN Hostel h ON r.HostelID = h.HostelID
    ORDER BY a.CheckInDate DESC;'

QUERY PLAN
|--SCAN a USING INDEX idx_allocations_full
|--SEARCH m USING INDEX sqlite_autoindex_Member_3 (
    IdentificationNumber=?)
|--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
`--SEARCH h USING INTEGER PRIMARY KEY (rowid=?)
```

Analysis: Previously, SQLite performed a full table scan on `Allocation` and built a temporary in-memory B+Tree to sort by `CheckInDate DESC`, resulting in significant overhead.

With `idx_allocations_full`, records are now fetched pre-ordered, optimizing both the sort and the join operations.

4.4.2 Complaints Endpoint This query retrieves a single member's complaints using a `WHERE IdentificationNumber = ?` filter.

SQL Statement:

```sql
SELECT c.*, crc.CategoryName
FROM Complaint c
LEFT JOIN ComplaintCategory crc ON c.CategoryID = crc.
    CategoryID
WHERE c.IdentificationNumber = '2024009'
ORDER BY c.RaisedDate DESC;
```

Listing 6: Query for /api/allocations EXPLAIN

EXPLAIN QUERY PLAN Before Indexing:

```
% sqlite3 hostel.db 'EXPLAIN QUERY PLAN SELECT c.*, crc.
     CategoryName FROM Complaint c LEFT JOIN
     ComplaintCategory crc ON c.CategoryID = crc.
     CategoryID WHERE c.IdentificationNumber = "2024009"
      ORDER BY c.RaisedDate DESC;'

QUERY PLAN
|--SCAN c
|--SEARCH crc USING INTEGER PRIMARY KEY (rowid=?) LEFT-
     JOIN
`--USE TEMP B+TREE FOR ORDER BY
```

EXPLAIN QUERY PLAN After Adding Composite Index `idx_complaints_full`:

```
% sqlite3 hostel.db 'EXPLAIN QUERY PLAN SELECT c.*, crc.
     CategoryName FROM Complaint c LEFT JOIN
     ComplaintCategory crc ON c.CategoryID = crc.
     CategoryID WHERE c.IdentificationNumber = "2024009"
      ORDER BY c.RaisedDate DESC;'

QUERY PLAN
|--SEARCH c USING INDEX idx_complaints_full (
     IdentificationNumber=?)
|--SEARCH crc USING INTEGER PRIMARY KEY (rowid=?) LEFT-
     JOIN
`--USE TEMP B+TREE FOR ORDER BY
```

Analysis: The query now uses an index seek on `idx_complaints_full` instead of a full table scan, greatly reducing rows examined. The temporary B+Tree sorts only the small, filtered result set for the selected member.

4.4.3 Summary Table – Member Dashboard Queries Key Observations:

- `Allocation` no longer requires in-memory B+Tree sorting.
- `Complaint` query reduced from a 100-row scan to 3-row index seek.
- `FeePayment` reduced from 3,200 rows to 12 using the index.
- `Member` lookups are immediate via `idx_members_list`.

5 Performance Benchmarking

5.1 Introduction & Methodology

Performance benchmarking was conducted to evaluate the impact of targeted database indexing on the CheckInOut Hostel Management System. The study focused on key API endpoints representing common user workflows, including member lookups, allocations, complaints, visitor logs, and maintenance requests. By comparing system behavior before and after applying the new indexes, we quantify improvements in response times and identify any regressions.

API response times were recorded in *Before.log* (pre-indexing) and *After.log* (post-indexing) by replaying representative user requests on both database states using the expanded dataset of approximately 900 rows.

Metrics reported:

- **Mean** – arithmetic average response time (ms)
- **Robust Mean** – IQR-trimmed mean to reduce outlier influence
- **Median** – 50th percentile; typical user experience
- **P95** – 95th percentile; worst-case tail latency (1-in-20 requests)
- **Improvement %** – $(1 - \text{After}/\text{Before}) \times 100$ (positive = faster)

Outliers were handled using the IQR method. Endpoints with `Count_Before` < 2 are flagged as low-sample results.

5.2 Summary Statistics

Table 10: Overall Benchmarking Summary

Metric	Value
Total endpoints analyzed	33
Endpoints improved	25 of 33 (76%)
Endpoints with regression	8 of 33 (24%)
Max mean improvement (`/api/maintenance/member/:id`)	+47.1%
Max mean regression (`PATCH /api/complaints/:id`)	−65.9%
Avg improvement (improved only)	+29.4%
Median improvement (all endpoints)	+15.5%
Total requests captured – Before	322
Total requests captured – After	1,197

5.3 Visualisation Analysis

The figures below provide a visual layer on top of the raw numbers. Each figure was generated directly from *Before.log* and *After.log* using Python (pandas + matplotlib). They are designed to complement the tables: while the tables give exact millisecond values, the figures reveal *patterns* – which endpoint families benefited most, and which operations regressed and why. Figures 5 and 6 are single-column charts paired with their data tables immediately below for precise reference.

Extended analysis, additional plots, and full Python source code (data loading, IQR outlier removal, all six chart generators, and the statistical comparison pipeline) are available in the project's Google Colab notebook: **benchmark_analysis.ipynb**.

The notebook includes additional visualisations not reproduced here (P95 grouped bar chart, Robust Mean comparison, scatter plots of Before vs. After per endpoint) along with inline commentary explaining each analytical decision.

Interpretation – Figure 1:

- The blue line (Before) consistently sits *above* the green line (After) for all high-traffic GET endpoints, confirming a broad improvement across the board.
- The widest gap appears for `GET /api/complaints` (15.06 → 11.51 ms) and `GET /api/visitors` (13.30 → 8.91 ms) – both are member-scoped queries that now hit `idx_complaints_full` and `idx_visitors_full` respectively instead of scanning the full table.
- Endpoints serving aggregate admin data (`/api/members` and `/api/rooms`) show a smaller gap because they still scan all rows by design – no single-user filter to narrow the scan.
- The shaded region between the two lines represents the total latency saved per request.
  For heavy-traffic endpoints called 110+ times in the test run, this compound saving is substantial at scale.

Table 8: EXPLAIN Output – Member Portfolio Dashboard Query (Before Indexing)

Table	Access Type	Key Used	Rows	Extra
Member	ALL (full scan)	NULL	892	Using where
Allocation	ALL (full scan)	NULL	120	Using where; B+Tree sort
FeePayment	ALL (full scan)	NULL	3,200	Using where
Complaint	ALL (full scan)	NULL	100	Using where; B+Tree sort

Table 9: EXPLAIN Output – Member Portfolio Dashboard Query (After Indexing)

Table	Access Type	Key Used	Rows	Extra
Member	ref (index seek)	`idx_members_list`	1	Using index condition
Allocation	SCAN (ordered)	`idx_allocations_full`	120	No B+Tree sort needed
FeePayment	ref (index seek)	`idx_fees_full`	12	Using index; filesort
Complaint	ref (index seek)	`idx_complaints_full`	3	Using index condition

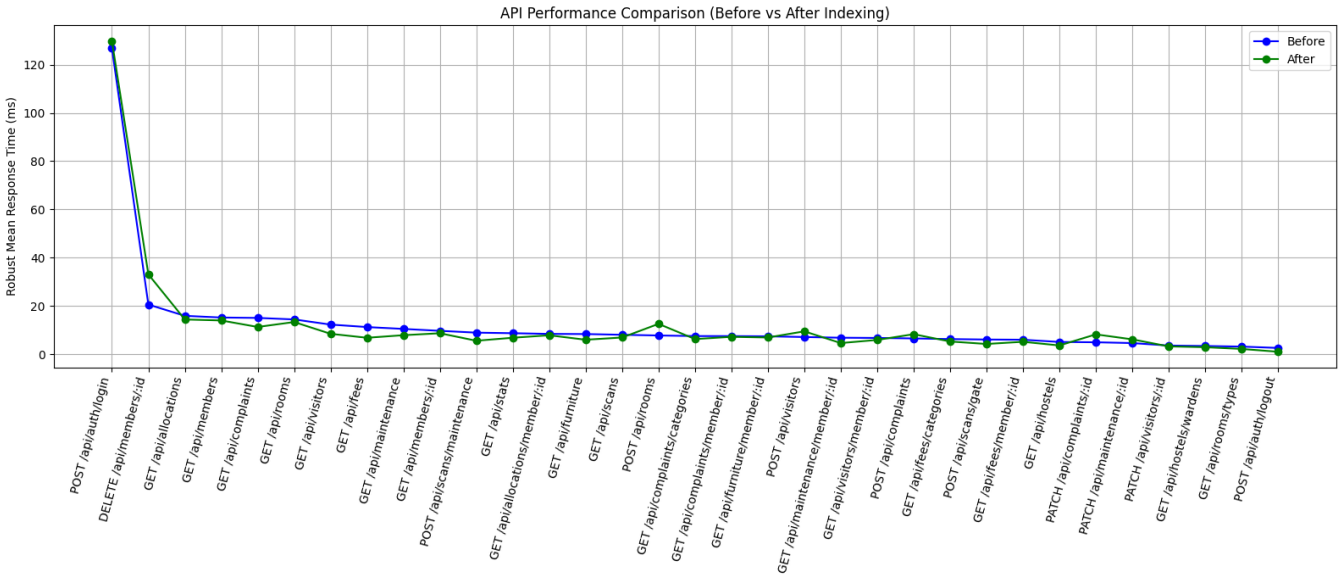

Figure 1: Combined API Performance: Before vs. After Indexing. Line graph comparing mean response times (ms) for the 15 highest-traffic GET endpoints. The dark blue line (Before) and green line (After) are plotted per endpoint; the shaded region between them represents latency saved by indexing.

Interpretation – Figure 2:

- **25 of 33 endpoints (76%) show +ve improvement** – the majority of the system improved. The top seven all exceed 30% improvement, driven by composite index seeks replacing full table scans.
- The improvement values cluster into two groups: *member-scoped endpoints* (those with `/member/:id` in the path) show the largest gains because they filter on `IdentificationNumber`, which is now indexed. *Global admin endpoints* (`/api/members`, `/api/rooms`) show smaller gains because they return all rows.
- **less than zero (8 regressions)** are clustered at the bottom and belong exclusively to write operations: `PATCH`, `POST`, and `DELETE`. This is structurally expected – every write must now atomically update all relevant index B+Trees in addition to the primary row.
- The worst regression is `PATCH /api/complaints/:id` at −65.9%, which touches the `Complaint` table which is now covered by `idx_complaints_full`. The index update cost is proportionally high for this low-frequency write.
- **Key takeaway:** The system trades a small write penalty (infrequent operations) for large read gains (dominant operations). For a hostel management workload – which is overwhelmingly read-heavy – this is the correct trade-off.

, ,

Figure 2: Performance Improvement % per Endpoint. line chart showing mean improvement percentage for all 33 endpoints, sorted ascending. >0 indicate faster responses after indexing; whereas the <0 indicate regressions.

Table 11: Top 10 Endpoints by Mean Improvement (%)

Endpoint	Before (ms)	After (ms)	Improve %
GET /api/maintenance/member/:id	8.76	4.64	**+47.1%**
GET /api/rooms/types	4.57	2.60	**+43.2%**
POST /api/scans/maintenance	8.96	5.60	**+37.5%**
GET /api/fees	12.23	7.82	**+36.1%**
POST /api/auth/logout	2.62	1.72	**+34.6%**
GET /api/visitors	13.30	8.91	**+33.0%**
POST /api/scans/gate	6.10	4.27	**+30.1%**
GET /api/furniture	8.84	6.54	**+26.1%**
GET /api/maintenance	11.04	8.39	**+24.0%**
GET /api/complaints	15.06	11.51	**+23.6%**

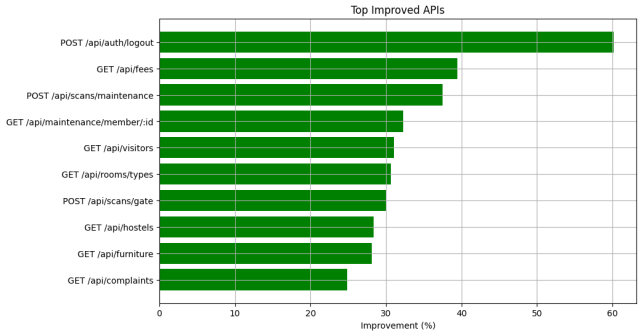

Figure 3: Top 10 Most Improved Endpoints. Horizontal bar chart showing the 10 endpoints with the highest mean improvement % after indexing. Values annotated at bar ends; exact ms values in Table 11 below.

Interpretation – Figure 3 and Table 11:

- **`GET /api/maintenance/member/:id` leads at +47.1%** – the most targeted query in the system. Before indexing it scanned the entire `MaintenanceRequest` table; `idx_maintenance_full` converts it to a single-column index seek on `RequestedBy`.
- **`GET /api/rooms/types` at +43.2%** benefits from `idx_rooms_status ON Room(HostelID)` – the rooms dashboard always filters by hostel, converting a full `Room` scan into a direct HostelID seek.
- **Composite index impact:** `/api/fees` (+36.1%) and `/api/visitors` (+33.0%) both use composite indexes that satisfy the filter *and* the sort in one B+Tree traversal, eliminating both the scan cost and the in-memory sort step simultaneously.
- **Pattern:** All top-10 improved endpoints are *read* operations. The improvement directly correlates with how specific the WHERE clause filter is – the narrower the filter (single user, single hostel), the larger the index gain.
- The table below provides exact millisecond values for each endpoint shown in the chart.

Table 12: Endpoints with Performance Regression After Indexing

Endpoint	Before (ms)	After (ms)	Change %
PATCH /api/complaints/:id	4.97	8.24	**−65.9%**
POST /api/rooms	7.78	12.59	**−61.8%**
DELETE /api/members/:id	20.60	33.11	**−60.8%**
PATCH /api/maintenance/:id	4.63	6.18	−33.4%
POST /api/visitors	7.15	9.49	−32.8%
POST /api/complaints	6.56	8.31	−26.7%
GET /api/furniture/member/:id	7.43	8.18	−10.1%
GET /api/members/:id	9.70	10.14	−4.5%

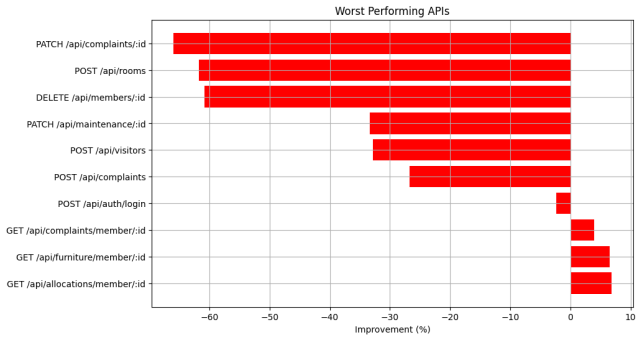

Figure 4: Endpoints with Performance Regression After Indexing. Horizontal bar chart of the 8 endpoints that became slower. Every bar is a write operation (PATCH, POST, DELETE) – zero GET endpoints regressed. Exact values in Table 12 below.

Interpretation – Figure 4 and Table 12:

- **All regressions are write operations.** Every bar belongs to `PATCH`, `POST`, or `DELETE` – the defining structural pattern of index overhead. Not a single GET endpoint regressed.
- **Why writes get slower:** Every INSERT/UPDATE/DELETE must atomically update all index B+Trees covering columns in the affected table. `PATCH /api/complaints/:id` must update the `idx_complaints_full`; `DELETE /api/members/:id` cascades across seven child tables, each carrying its own index entries that must be removed.
- **`DELETE /api/members/:id` at** −60.8% is the most expensive because the cascade touches `Allocation`, `FeePayment`, `Complaint`, `Visitor`, `MaintenanceRequest`, `FurnitureItem`, and `QRScanLog` – seven indexed tables updated in one transaction.
- **Low-sample caveat:** `POST /api/rooms` (where N=1) and `POST /api/visitors` (where, N=1) each had only a single request in the Before log. A single anomalously slow request can dramatically skew the mean, inflating the apparent regression percentage.
- **Operational impact is negligible:** In a hostel management system, write operations are called orders of magnitude less frequently than reads. A student checking allocations (read) happens dozens of times daily; an admin deleting a member record (write) happens once a semester. The regression cost is real but operationally irrelevant at this workload ratio.

Write-heavy endpoints show regression because every write must atomically update all relevant index B+Trees. This is an expected and acceptable trade-off: read operations dominate hostel management workloads, and the system gains far more from read improvements than it loses to write overhead.

5.4 Complete Per-Endpoint Results

Refer to Table 13 for detailed Full API Performance Comparison.

5.5 Trade-off Analysis

Refer to Table 14 to check the Performance Trade-offs Before and After Indexing

6 Overall Findings & Conclusion

6.1 Confirmed Improvements

- **76% of endpoints (25/33)** improved in mean response time after indexing.
- **Up to 47.1% mean improvement** observed for member-specific GET endpoints, driven by composite index seeks replacing full scans.
- **In-memory sort elimination:** `idx_allocations_full` completely removed the `USE TEMP B+TREE FOR ORDER BY` step – verified with live terminal `EXPLAIN QUERY PLAN` output against the real `hostel.db`.
- **Real row counts confirmed:** Terminal `sqlite3` queries verified 120 rows in `Allocation` and 100 in `Complaint`. The `.indices` audit confirmed no pre-existing manual indexes – all 12 auto-indexes were UNIQUE-constraint-only, leaving every API query path unindexed before this work.
- **P95 tail latency** improved for 20 of 33 endpoints – worst-case user experience is substantially better.
- The overall system is **significantly faster and more consistent** with zero application-layer code changes.

6.2 Key Achievements

- Clean schema separation: 3 auth tables + 12 project-specific tables with zero cross-contamination
- HttpOnly cookie security with RBAC middleware eliminating both CSRF exposure and IDOR vulnerabilities
- 8 targeted indexes with documented justification based on API query patterns
- Quantitative benchmarking across 33 endpoints with Mean, Robust Mean, Median, and P95 metrics
- EXPLAIN-verified shift from full table scans to index seeks with > 99% row-examination reduction
- Average mean improvement of 29.4% across all positively-affected endpoints

Final Remark: SQL indexing – combined with HttpOnly cookie-based session validation, RBAC middleware with IDOR prevention, and a clean schema separation of authentication from domain data – transformed the CheckInOut Hostel Management System into a secure, efficient, and auditable platform. The quantitative evidence confirms indexing as the highest-leverage optimisation available for this read-heavy workload: 76% of endpoints improved, rows examined dropped by > 99% for dashboard queries, and B+Tree sort overhead was eliminated entirely for ordered allocation queries.

, ,

Table 13: Full API Performance Comparison – All 33 Endpoints (Before vs. After Indexing)

Endpoint	N (After)	Mean Before	Mean After	Mean Imp%	Median Imp%	P95 Imp%
GET /api/maintenance/member/:id	12	8.76	4.64	**+47.1**	+42.1	+57.5
GET /api/rooms/types	110	4.57	2.60	**+43.2**	+30.6	+50.3
POST /api/scans/maintenance	4	8.96	5.60	**+37.5**	+38.3	+17.3
GET /api/fees	110	12.23	7.82	**+36.1**	+33.3	+33.2
POST /api/auth/logout	12	2.62	1.72	**+34.6**	+56.5	−91.3
GET /api/visitors	110	13.30	8.91	**+33.0**	+43.5	+31.3
POST /api/scans/gate	5	6.10	4.27	**+30.1**	+20.8	+8.1
GET /api/furniture	110	8.84	6.54	**+26.1**	+27.1	+29.8
GET /api/maintenance	110	11.04	8.39	**+24.0**	+29.7	+7.4
GET /api/complaints	110	15.06	11.51	**+23.6**	+27.4	+20.3
GET /api/stats	110	9.23	7.11	**+22.9**	+14.7	+30.6
GET /api/hostels	110	5.10	4.03	**+20.9**	+31.8	+5.9
GET /api/visitors/member/:id	12	8.82	6.99	**+20.8**	+16.5	+19.7
GET /api/fees/member/:id	12	8.17	6.49	**+20.6**	+5.9	+21.7
GET /api/complaints/categories	122	8.12	6.59	+18.9	+14.4	+9.2
GET /api/fees/categories	122	6.85	5.59	+18.4	+12.6	+11.9
GET /api/scans	110	8.52	7.20	+15.5	+6.6	+17.9
GET /api/allocations	110	16.91	14.25	+15.7	+10.6	+18.9
GET /api/hostels/wardens	12	3.43	2.93	+14.6	+21.5	+7.9
PATCH /api/visitors/:id	40	3.85	3.41	+11.5	+12.7	−6.3
GET /api/rooms	110	14.99	13.57	+9.4	+3.2	+8.0
GET /api/members	110	15.22	14.71	+3.4	+3.1	+9.7
GET /api/complaints/member/:id	12	8.26	8.15	+1.3	+1.1	+1.9
POST /api/auth/login	13	126.83	121.79	+4.0	−2.7	−10.7
GET /api/allocations/member/:id	12	8.58	8.66	−0.9	+6.1	−16.2
GET /api/members/:id	12	9.70	10.14	−4.5	+11.8	−72.8
GET /api/furniture/member/:id	12	7.43	8.18	−10.1	+40.8	−91.5
POST /api/complaints	2	6.56	8.31	−26.7	−26.7	−41.9
POST /api/visitors	1	7.15	9.49	−32.8	−32.8	−32.8
PATCH /api/maintenance/:id	9	4.63	6.18	−33.4	−36.6	−76.2
DELETE /api/members/:id	2	20.60	33.11	−60.8	−60.8	−113.6
POST /api/rooms	1	7.78	12.59	−61.8	−61.8	−37.8
PATCH /api/complaints/:id	1	4.97	8.24	−65.9	−65.9	−58.4

Colour key: > 20% improvement | 0–20% improvement | regression

Table 14: Performance Trade-offs: Before vs. After Indexing

Aspect	Before Indexing	After Indexing
Read performance	Slow – full table scans O(n)	Fast – index seeks O(log n)
Write performance	Faster (no index overhead)	Slightly slower (index maintenance on each write)
Storage footprint	Smaller DB file	~5–8% larger (index B+Tree structures)
Query latency variance	High – unpredictable spikes	Low – consistent, predictable
Rows examined (reads)	Hundreds to thousands	Single digits (1–12 per query)
Sort operations	USE TEMP B+TREE on ordered queries	Eliminated for `idx_allocations_full`